# Proyecto. Árboles binarios simples y árboles Rojo-Negros.

Disponible: 13 de Marzo.

Entrega: 17 de Abril 23:55.

En este proyecto se va a trabajar con la estructura de árboles binarios de búsqueda (con su ampliación de árboles rojo y negros).

# 1 Instrucciones de entrega

El proyecto podrá realizarse por parejas o de forma individual (pero no estará pertmitido generar grupos de 3 personas). Cada pareja deberá avisar a su correspondiente profesor de teoría sobre los miembros del equipo y deberá entregar un fichero.zip conteniendo todos los ficheros Python requeridos del proyecto. El formato de entrega será el siguiente:

```
<nombreAlumno1>_<ApellidoA1>_<ApellidoA2>_
<nombreAlumno2>_<ApellidoA2>_<ApellidoA2>_Proyecto.zip
```

Por ejemplo:

```
Pablo_DelasHeras_Fernandez_Marina_Perez_Gonzalez_Proyecto.zip
```

**IMPORTANTE: Recuerda que la funcionalidad del programa es crítica, al igual que un correcto estilo de código y documentación. Por esa razón, código que no esté correctamente documentado y no tenga un estilo correcto, será penalizado**.

**IMPORTANTE: En caso de realizar el proyecto por parejas, se deberá entregar una memoria/informe/documento.pdf en el que se especifique qué tareas ha realizado cada miembro de la pareja.**

**Es posible que si programáis funciones/métodos recursivos, a lo largo del proyecto os devuelva un error de recursión infinita aun estando el código bien. Para evitarlo, podéis poner esta sentencia al inicio de vuestro código:**

```
sys.setrecursionlimit(15000)
```

# 2 Primera parte: árboles binarios de búsqueda.

Deberás crear un fichero llamado BinaryTree.py que contenga la definición de los nodos y los árboles binarios de búqueda especificados en la clase de laboratorio. Dichero fichero deberá contener los siguientes métodos:

- insert: deberá insertar un nodo con la clave y el valor.

  ```
  def insert(self, key, value=None):
  ```

- show: deberá imprimir el árbol con la clave y el valor de cada uno de los nodos:

  ```
  def show(self):
  ```

- height: debe devolver la altura del árbol.

```
def height(self):
```

- find_node: deberá buscar en el árbol la clave del nodo y devolver la pareja de clave valor si está y None en caso contrario.

```
def find_node(self, key):
```

- delete: deberá eliminar del árbol el nodo con la la clave key.

```
def delete(self, key):
```

- skew: define un método de la clase de árbol llamado skew que compruebe si un árbol está inclinado o no. Recordad que un nodo está inclinado si su balance no es 0, 1 o -1:

```
def skew(self) ->bool:
```

## 2.1 Diferentes formas de imprimir un árbol

Añade el código de la práctica para poder imprimir el árbol en estas 4 configuraciones:

- In-order: Este recorrido visita el subárbol izquierdo, luego el nodo raíz y, por último, el subárbol derecho. El prototipo de la función puede ser el siguiente:

```
def in_order(tree: BinaryTree) ->None:
```

- Pre-order: Este recorrido visita primero el nodo raíz, seguido del subárbol izquierdo y, a continuación, el subárbol derecho. El prototipo de la función puede ser el siguiente:

```
def pre_order(tree: BinaryTree) ->None:
```

- Post-order: Este recorrido visita el subárbol izquierdo, luego el subárbol derecho y, por último, el nodo raíz.

```
def post_order(tree: BinaryTree) ->None:
```

- LevelOrder: Este recorrido se hace por niveles. Arriba-abajo e Izq. a Dcha.

```
def level_order(tree: BinaryTree) ->None:
```

# 3 Segunda parte: Lectura de palabras

Se dispone de un fichero llamado palabras_RAE_cleaned_noaccents.txt que contiene todas las palabras de la RAE ordenadas alfabéticamente, de forma que en cada línea hay una palabra. Se pide crear un fichero llamado Proyecto.py distinto al fichero donde esté la definición de los árboles binarios de búsqueda. Dicho fichero Proyecto.py, contendrá el programa principal del proyecto, que deberá realizar las siguientes tareas:

- Preguntar por pantalla hasta 3 letras.
- Por cada una de las letras introducidas por teclado, se pide almacenar todas las palabras que hay en el fichero palabras_RAE_cleaned_noaccents.txt que comiencen por esa letra en un árbol binario de búsqueda.
- Una vez leídas cada una de las letras, se pide mostrar los siguientes datos en los tres árboles creados:
  - La palabra que está en el nodo raíz.
  - La palabra que está en el nodo de la izquierda del nodo raíz.
  - La palabra que está en el nodo de la derecha del nodo raíz.
  - Calcular el número de nodos (palabras) del árbol. Deberás crear un método en la clase BinaryTree siguiendo este prototipo.

    ```
    def count_nodes_tree(self) ->int:
    ```

  - Contar el número de palabras que están en nodos hoja. Deberás crear un método de la clase de BinaryTree siguiendo este prototipo:

    ```
    def count_leafs(self) ->int:
    ```

  - Las palabras que conforman la ruta más larga desde las hojas hasta la palabra de la raíz. Deberás crear un método en la clase BinaryTree siguiendo este prototipo:

    ```
    def longest_path(self) ->list:
    ```

  - Una función para devolver todos los caminos raíz->hoja con longitud exacta (en aristas) igual a remaining_edges al comienzo de la llamada. Deberás aplicar backtracking para resolver este ejercicio. Sigue este prototipo:

    ```
    def paths_to_leaf_with_length(
        node: Optional["Node"],
        remaining_edges: int,
        current_path: Optional[List[str]] =None,
        results: Optional[List[List[str]]] =None,
    ) ->List[List[str]]:
    ```

    La descripción más detallada de la función es la siguiente:

```
"""
    Devuelve todos los caminos raiz->hoja con longitud exacta (en
                                                aristas) igual
    a remaining_edges al comienzo de la llamada.

    Uso tipico:
        paths = paths_to_leaf_with_length(tree.root, 3)

    Parametros:
    - node: nodo actual (comienza en la raiz)
    - remaining_edges: cuantas aristas faltan para alcanzar la
                                                longitud objetivo
    - current_path: (interno) lista con el camino actual
    - results: (interno) acumulador de caminos validos

    Retorna:
    - Lista de caminos; cada camino es una lista de strings (str(node.
                                                key)).
 """
```

Te habrás fijado en que estás obteniendo valores un tanto "peculiares" en algunas de las funciones anteriores. ¿Sabrías determinar por qué? Modifica el código desarrollado para permitir al usuario seleccionar si quiere la inserción por defecto o si prefiere una inserción aleatoria. La inserción por defecto consiste en coger las palabras tal cual aparecen en el fichero de palabras de la rae (es decir, línea por línea). La inserción aleatoria consiste en añadir todas las palabras que comiencen por la letra pedida, meterlas en una lista de Python y desordenarlas de forma aleatoria. Luego, deberás insertar palabra a palabra siguiendo ese orden aleatorio en cada uno de los árboles.

# 4 Tercera parte: Programación de los árboles Rojo negros.

Continuando con la definición de los árboles rojo-negros que se había descrito en el laboratorio se pide implementar todos los métodos anteriores para la clase de árbol binario de búsqueda pero para la clase de árboles rojo-negros (es decir, los métodos contar los nodos, si está balanceado o no, media de profundidad, etc, además de todos los métodos de inserción, búsqueda, eliminación, impresión, etc). Recuerda que en este caso, seguramente necesites implementar la función de ayuda con la eliminación (fix-up).

```
def delete_fixup(self, x: RBNode) ->None:
```

Amplía el código para leer las palabras de la rae que comienzan con las tres letras elegidas, de forma que se creen 2 árboles para cada letra. Una con un árbol binario de búsqueda normal y otra con el árbol rojo-negro. Compara los valores obtenidos en las funciones con cada uno de ellos.

## 5 Opcional

Añade empleando la librería que quieras (pero especificando cuál has utilizado) para hacer plots con los árboles generados. Es decir, se debe permitir generar gráficos con los árboles creados. Debes enseñar los plots en diferentes casos de pruebas.

## ENTREGA

Un miembro de cada pareja (solo 1) deberá entregar los siguientes ficheros:

- BinaryTree.py: conteniendo toda la estructura de los árboles binarios de búsqueda, junto con un main de prueba que incluya inserciones y eliminaciones de algunos elementos. Para cada prueba, se debe especificar claramente la salida esperada y la salida obtenida con vuestra implementación. Haz lo mismo para cada una de las funciones de impresión del árbol (in_order, pre_order, post_order, level_order) y también prueba las funciones de contar el número de nodos de un árbol, si está balanceado, media de profundidad del árbol, longitud máxima, etc.

- RedBlackTree.py: conteniendo toda la estructura de los árboles rojo-negros, junto con un main de prueba que incluya inserciones y eliminaciones de algunos elementos. Para cada prueba, se debe especificar claramente la salida esperada y la salida obtenida con vuestra implementación. Haz lo mismo para cada una de las funciones de impresión del árbol (in_order, pre_order, post_order, level_order) y también prueba las funciones de contar el número de nodos de un árbol, si está balanceado, media de profundidad del árbol, longitud máxima, etc.

- Proyecto.py: funcionalidad relativa a la segunda parte, incluyendo el menú y la posibilidad de leer letras por teclado para mostrar todos los detalles de las palabras asociadas a cada letra.

- Memoria/informe/documento especificando qué tareas/funciones ha realizado cada persona de la pareja.

- Cualquier fichero adicional para hacer funcionar el proyecto donde hayáis definido funciones auxiliares. No es necesario entregar el fichero de texto conteniendo las palabras.